

GraphCompose Markdown

A themeable Markdown ? PDF document composer

Artem Demchyshyn · 2026-06-21

What this is

graphcompose-markdown turns Markdown into a fully themed PDF. It parses Markdown with [Flexmark](#), maps it onto an independent semantic model, and renders that model through a swappable theme into the [GraphCompose](#) layout engine.

Note

This page is Markdown — rendered to PDF by the library it describes. Everything you see is a feature: the title block above, this alert, the table below, the footnote.^[1]

It is not a plain converter: content (Markdown), appearance (theme) and layout (the engine) stay separate, so the same text can be reskinned without editing it. The project lives at <https://github.com/DemchaAV/graphcompose-markdown>.

Why use it

- Separation of concerns — content, theme and layout are independent.
- Parser-decoupled model — renderers never see Flexmark, so the parser is swappable.
- Composable renderers — override one node type, ship a whole pack, or add a `:::` block.
- Real document features — a navigable PDF outline, footnotes, tables and alerts.

Inline formatting

A paragraph can carry bold, italic, bold italic, strikethrough and `inline code`. Links are explicit — [the repository](#) — or bare and auto-linked, like <https://central.sonatype.com>. Emoji shortcodes resolve to inline images: ship it 🚀, tests pass ✅, celebrate 🎉.

Lists and tasks

Unordered lists nest by depth:

- Parse — Flexmark plus the GFM extensions
 - tables, task lists, strikethrough, footnotes
 - emoji, autolinks, YAML front matter
- Map — a sealed `MarkdownNode` tree
- Render — one `NodeRenderer` per node type

Ordered lists keep their numbers:

1. Parse the Markdown

2. Map to the semantic model
3. Render through the theme

Task lists track progress:

- ☒ Tables, alerts, footnotes, custom blocks
- ☒ Syntax highlighting, emoji, autolinks, front matter
- ☐ A DOCX backend (the engine already supports it)

Code

Fenced code is syntax-highlighted by a pluggable highlighter (~15 languages, no extra dependency):

```
var composer = MarkdownComposer.builder()  
    .theme(DefaultMarkdownTheme.light())  
    .build();  
// the three lines that produced this very document:  
composer.render(markdown).writePdf(Path.of("showcase.pdf"));
```

```
def render(markdown: str) -> bytes:  
    # the same idea in another language's highlighting  
    return compose(markdown).to_pdf_bytes()
```

Tables

GFM tables support per-column alignment — left, centre and right:

Feature	Status	Since
Headings to outline	stable	0.1.0
Syntax highlighting	stable	0.1.0
GitHub-style alerts	new	0.1.0
Emoji shortcodes	new	0.1.0

Alerts

GitHub-style alerts render as titled, colour-coded callouts:

 **Note**

Useful information that users should know.

 **Tip**

Helpful advice for doing things better.

Important

Key information users need to succeed.

Warning

Urgent info that needs immediate attention.

Caution

Advises about a risk or a negative outcome.

Quotes, rules and custom blocks

A blockquote is a left-accented aside. Themes decide how it looks — design tokens for the cosmetics, node renderers for the behaviour.

A `:::` custom block routes to a registered renderer, falling back to a callout:

Derive a theme from an existing one with `MarkdownTheme.builder(base)` and override only what differs — every other component is reused.

A reskinned render

The same Markdown, restyled by the GitHub-dark theme pack:

Quarterly Review

A themeable Markdown composer — the same content, different themes.

- Revenue up
- Costs down
- [Details](#)

Design tokens drive the cosmetics; renderers drive the behaviour.

Metric	Q3	Q4
Revenue	120	140
Margin	18%	22%

```
MarkdownTheme theme = GitHubTheme.light();
```

Derive a pack from the defaults and override only what differs.

Architecture

```
Markdown --Flexmark--> AST --mapper--> MarkdownNode tree
--theme + renderers--> GraphCompose model --engine--> PDF
```

The semantic model is the stable hand-off point: the parser is swappable, renderers operate on the model (never on Flexmark types), and you can build or transform the model directly.

Heading levels

This document also exercises every heading level.

Level four

Level five

Level six

Notes

[1] Footnotes are collected into this Notes section; the reference above links here by number, and the engine keeps the numbering correct across the whole document.